

Nama: Fabyan Yastika Permana
NIM: 242410101041
Kelas: A

AKTIVITAS MANDIRI 3 — Membuat Layout Master & Halaman-Halaman

□ AKTIVITAS MANDIRI 3 — Membuat Layout Master & Halaman-Halaman

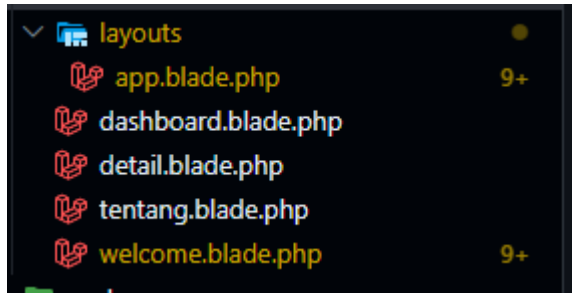
1. Buat layout master: resources/views/layouts/app.blade.php dengan navbar, main content area (@yield('content')), dan footer
2. Buat partial: resources/views/partials/navbar.blade.php dan sisipkan dengan @include di layout
3. Buat halaman dashboard: resources/views/dashboard.blade.php yang @extends('layouts.app')
4. Tampilkan flash session message di layout: @if(session('success')) ... @endif
5. Buat komponen: resources/views/components/stat-card.blade.php (kartu statistik dengan props: judul, nilai, ikon, warna)
6. Gunakan @forelse di dashboard untuk loop array data dummy (jumlah mahasiswa, mata kuliah, dll)
7. Buat halaman 'Tentang' dan 'Kontak' yang @extends layout yang sama
8. Gunakan @push('scripts') dan @stack('scripts') untuk menambahkan JavaScript spesifik per halaman

□ *Tips: Gunakan @include('partials.nama') untuk memecah tampilan menjadi komponen kecil yang reusable. Ini membuat kode View lebih mudah dikelola.*

1. Buat layout master: resources/views/layouts/app.blade.php dengan navbar, main content area (@yield('content')), dan footer

```
resources > views > layouts > @ app.blade.php > @test:scroll-smooth > @head
1 <doctype html>
2 <html lang="id" class="scroll-smooth">
3
4 <head>
5     <meta charset="UTF-8" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>@yield('title', 'Libmate | Pemesanan Ruang Perpus AI')</title>
8
9     <!-- Scripts & Styles -->
10    @vite(['resources/css/app.css', 'resources/js/app.js'])
11
12    <!-- Fonts -->
13    <link href="https://fonts.googleapis.com/css2?family=PlusJakartaSans:wght@300;400;500;600;700;800&display=swap"
14          rel="stylesheet" />
15    <link rel="shortcut icon" href="{{ asset('libmate.png') }}" type="image/x-icon" />
16
17    <!-- Include Icons -->
18    <script src="https://unpkg.com/ionicons@latest"></script>
19
20    <!-- Tailwind Configuration (Jika tidak menggunakan build tool penuh) -->
21    <script>...
22    </script>
23    </head>
24
25    <body class="bg-slate-50 @test:slate-800 antialiased overflow-x-hidden">
26    <!-- Navbar Component -->
27    <nav>
28
29    </nav>
30
31    <!-- Main Content Area -->
32    <main>
33        @yield('content')
34    </main>
35
36    <!-- Footer Component -->
37    <footer class="bg-slate-50 pt-16 pb-8 border-t border-slate-200 @test:slate-600">
38
39    </footer>
40
41    <div class="fixed bottom-6 right-6 z-[100] flex flex-col items-end">
42
43    </div>
44
45    <!-- Scripts stack for page-specific JS -->
46    @stack('scripts')
47    </script>
48    @stack('scripts')
49    </script>
50    </body>
51    </html>
```

Bagian ini merupakan "dasar" dari seluruh interface web LIBMATE. Di sini, saya mendefinisikan struktur HTML5 standar yang mencakup meta tags untuk responsivitas dan pemanggilan aset utama melalui @viteIntinya seperti perintah soal yaitu @yield('content'), yang berfungsi sebagai kontainer dinamis. Baris ini memerintahkan Laravel untuk menyediakan ruang kosong yang nantinya akan diisi oleh konten dari setiap halaman child. Selain itu, terdapat @stack('scripts') di bagian bawah untuk menambahkan JavaScript spesifik halaman tanpa campur dengan struktur global.

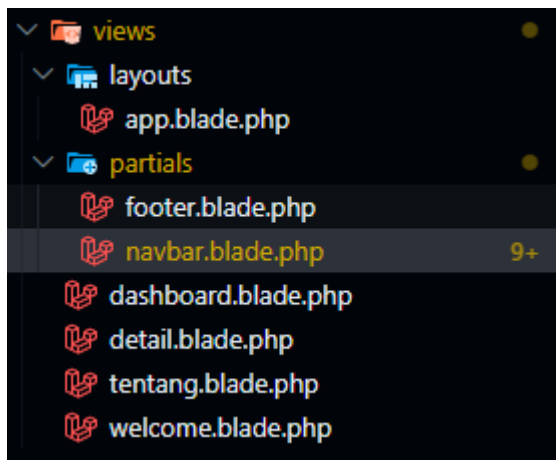


Saya mengelola arsitektur file di dalam folder resources/views/layouts/. Saya memisahkan antara app.blade.php untuk halaman publik dan dashboard.blade.php untuk interface admin agar pengelolaan kode lebih modular. File-file halaman seperti welcome, tentang, dan detail tidak perlu lagi menulis kode header atau footer dari awal. Mereka cukup "meminjam" kerangka dari layout master tersebut, sehingga jika ada perubahan pada navigasi, saya hanya perlu mengubah satu file saja untuk mengupdate seluruh website.



Ini adalah visualisasi bagaimana sistem templating bekerja. Halaman ini adalah hasil penggabungan antara kerangka statis dari Layout Master (Navbar dan Footer) dengan konten dinamis yang didefinisikan di welcome.blade.php. Semua komponen visual, mulai dari logo Universitas Jember hingga tombol aksi "Pesan Ruangan", muncul dengan rapi karena CSS yang dimuat di layout master berhasil diaplikasikan secara menyeluruh.

2. Buat partial: resources/views/partials/navbar.blade.php dan sisipkan dengan @include di layout



Di sini saya menerapkan konsep Partials untuk memecah komponen antarmuka yang statis. File navbar.blade.php dan footer.blade.php dipisahkan ke dalam folder khusus agar kode lebih rapi dan mudah dikelola tanpa mengotori file layout utama.

<pre> 1 <doctype html> 2 <html lang="id" class="scroll-smooth"> 3 4 <head> 5 <meta charset="UTF-8" /> 6 <meta name="viewport" content="width=device-width, initial-scale=1" /> 7 <title>@yield('title', 'LIBMTC Pemesanan Ruang Perpustakaan')</title> 8 9 <!-- Scripts & Styles --> 10 @vite(['resources/css/app.css', 'resources/js/app.js']) 11 12 <!-- Fonts --> 13 <link href="https://fonts.googleapis.com/css2?family=PlusJakarta+Sans:wght@300;400;500;600;700;800&display=swap" 14 rel="stylesheet" /> 15 <link rel="shortcut icon" href="{{ asset('libmate.png') }}" type="image/x-icon" /> 16 17 <!-- Lucide Icons --> 18 <script src="https://unpkg.com/lucide@latest"></script> 19 20 <!-- Tailwind Configuration (jika tidak menggunakan build tool penuh) --> 21 <script> 22 </script> 23 </script> 24 @stack('styles') 25 </head> 26 27 <body class="bg-slate-50 @text-slate-800 antialiased overflow-x-hidden"> 28 <!-- Navbar Component --> 29 @include('partials.navbar') 30 31 <!-- Main Content Area --> 32 @yield('content') 33 </main> 34 35 <!-- Footer Component --> 36 @include('partials.footer') 37 38 <!-- Scripts stack for page-specific JS --> 39 @stack('scripts') 40 <script> 41 lucide.createIcons(); 42 </script> 43 </body> 44 </html> </pre>	Ini menunjukkan proses penyatuan komponen menggunakan direktif <code>@include</code>. Saya memanggil file <code>partials</code> tersebut ke dalam <code>app.blade.php</code> sehingga navigasi dan footer otomatis muncul di setiap halaman yang menggunakan layout ini. Area konten tetap menggunakan <code>@yield('content')</code> untuk fleksibilitas isi halaman.
	Hasil akhirnya adalah halaman landing yang terintegrasi secara sempurna. Navbar di bagian atas dan Footer di bagian bawah (jika di-scroll) merupakan hasil dari <code>partials</code> yang dipanggil tadi, sementara bagian tengah (banner Universitas Jember) adalah konten dinamis yang mengisi area <code>@yield</code>.

3. Buat halaman dashboard: `resources/views/dashboard.blade.php` yang `@extends('layouts.app')`

<pre> views ├── layouts │ └── dashboard │ ├── app.blade.php │ └── app.blade.php ├── partials │ ├── dashboard │ │ ├── right-sidebar.blade.php │ │ ├── sidebar.blade.php │ │ ├── footer.blade.php │ │ ├── navbar.blade.php │ │ ├── dashboard.blade.php │ │ ├── detail.blade.php │ │ ├── tentang.blade.php │ │ └── welcome.blade.php </pre>	Saya mengorganisir file dashboard dengan memisahkan layout utama dan komponen <code>partials</code> ke dalam sub-folder khusus agar struktur proyek tetap rapi dan modular.
--	--

```

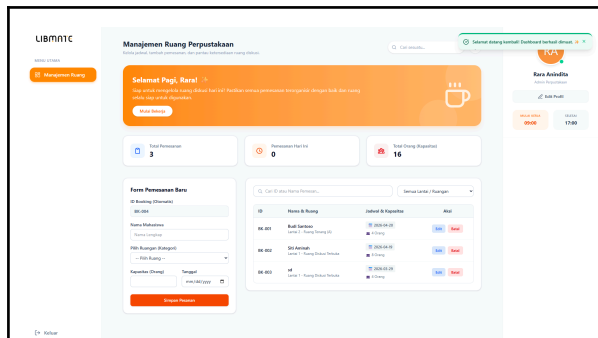
1 @app.directive('scrollbar')
2 @component
3 scrollbar
4 {
5   @input height: 0px;
6 }
7
8 @component
9 scrollbar
10 {
11   ::webkit-scrollbar-track {
12     background: transparent;
13   }
14
15   ::webkit-scrollbar-thumb {
16     background: #888888;
17     border-radius: 5px;
18   }
19
20   ::webkit-scrollbar-thumb:hover {
21     background: #555555;
22   }
23
24   ::webkit-scrollbar-thumb:horizontal {
25     background: #888888;
26   }
27
28   ::webkit-scrollbar-thumb:vertical {
29     background: #888888;
30   }
31
32   ::webkit-scrollbar-thumb:horizontal {
33     background: #888888;
34   }
35
36   ::webkit-scrollbar-thumb:vertical {
37     background: #888888;
38   }
39
40   ::webkit-scrollbar-thumb:horizontal {
41     background: #888888;
42   }
43
44   ::webkit-scrollbar-thumb:vertical {
45     background: #888888;
46   }
47
48   ::webkit-scrollbar-thumb:horizontal {
49     background: #888888;
50   }
51
52   ::webkit-scrollbar-thumb:vertical {
53     background: #888888;
54   }
55
56   ::webkit-scrollbar-thumb:horizontal {
57     background: #888888;
58   }
59
60   ::webkit-scrollbar-thumb:vertical {
61     background: #888888;
62   }
63
64   ::webkit-scrollbar-thumb:horizontal {
65     background: #888888;
66   }
67
68   ::webkit-scrollbar-thumb:vertical {
69     background: #888888;
70   }
71
72   ::webkit-scrollbar-thumb:horizontal {
73     background: #888888;
74   }
75
76   ::webkit-scrollbar-thumb:vertical {
77     background: #888888;
78   }
79
80   ::webkit-scrollbar-thumb:horizontal {
81     background: #888888;
82   }
83
84   ::webkit-scrollbar-thumb:vertical {
85     background: #888888;
86   }
87
88   ::webkit-scrollbar-thumb:horizontal {
89     background: #888888;
90   }
91
92   ::webkit-scrollbar-thumb:vertical {
93     background: #888888;
94   }
95
96   ::webkit-scrollbar-thumb:horizontal {
97     background: #888888;
98   }
99
100   ::webkit-scrollbar-thumb:vertical {
101     background: #888888;
102   }
103
104   ::webkit-scrollbar-thumb:horizontal {
105     background: #888888;
106   }
107
108   ::webkit-scrollbar-thumb:vertical {
109     background: #888888;
110   }
111
112   ::webkit-scrollbar-thumb:horizontal {
113     background: #888888;
114   }
115
116   ::webkit-scrollbar-thumb:vertical {
117     background: #888888;
118   }
119
120   ::webkit-scrollbar-thumb:horizontal {
121     background: #888888;
122   }
123
124   ::webkit-scrollbar-thumb:vertical {
125     background: #888888;
126   }
127
128   ::webkit-scrollbar-thumb:horizontal {
129     background: #888888;
130   }
131
132   ::webkit-scrollbar-thumb:vertical {
133     background: #888888;
134   }
135
136   ::webkit-scrollbar-thumb:horizontal {
137     background: #888888;
138   }
139
140   ::webkit-scrollbar-thumb:vertical {
141     background: #888888;
142   }
143
144   ::webkit-scrollbar-thumb:horizontal {
145     background: #888888;
146   }
147
148   ::webkit-scrollbar-thumb:vertical {
149     background: #888888;
150   }
151
152   ::webkit-scrollbar-thumb:horizontal {
153     background: #888888;
154   }
155
156   ::webkit-scrollbar-thumb:vertical {
157     background: #888888;
158   }
159
160   ::webkit-scrollbar-thumb:horizontal {
161     background: #888888;
162   }
163
164   ::webkit-scrollbar-thumb:vertical {
165     background: #888888;
166   }
167
168   ::webkit-scrollbar-thumb:horizontal {
169     background: #888888;
170   }
171
172   ::webkit-scrollbar-thumb:vertical {
173     background: #888888;
174   }
175
176   ::webkit-scrollbar-thumb:horizontal {
177     background: #888888;
178   }
179
180   ::webkit-scrollbar-thumb:vertical {
181     background: #888888;
182   }
183
184   ::webkit-scrollbar-thumb:horizontal {
185     background: #888888;
186   }
187
188   ::webkit-scrollbar-thumb:vertical {
189     background: #888888;
190   }
191
192   ::webkit-scrollbar-thumb:horizontal {
193     background: #888888;
194   }
195
196   ::webkit-scrollbar-thumb:vertical {
197     background: #888888;
198   }
199
200   ::webkit-scrollbar-thumb:horizontal {
201     background: #888888;
202   }
203
204   ::webkit-scrollbar-thumb:vertical {
205     background: #888888;
206   }
207
208   ::webkit-scrollbar-thumb:horizontal {
209     background: #888888;
210   }
211
212   ::webkit-scrollbar-thumb:vertical {
213     background: #888888;
214   }
215
216   ::webkit-scrollbar-thumb:horizontal {
217     background: #888888;
218   }
219
220   ::webkit-scrollbar-thumb:vertical {
221     background: #888888;
222   }
223
224   ::webkit-scrollbar-thumb:horizontal {
225     background: #888888;
226   }
227
228   ::webkit-scrollbar-thumb:vertical {
229     background: #888888;
230   }
231
232   ::webkit-scrollbar-thumb:horizontal {
233     background: #888888;
234   }
235
236   ::webkit-scrollbar-thumb:vertical {
237     background: #888888;
238   }
239
240   ::webkit-scrollbar-thumb:horizontal {
241     background: #888888;
242   }
243
244   ::webkit-scrollbar-thumb:vertical {
245     background: #888888;
246   }
247
248   ::webkit-scrollbar-thumb:horizontal {
249     background: #888888;
250   }
251
252   ::webkit-scrollbar-thumb:vertical {
253     background: #888888;
254   }
255
256   ::webkit-scrollbar-thumb:horizontal {
257     background: #888888;
258   }
259
260   ::webkit-scrollbar-thumb:vertical {
261     background: #888888;
262   }
263
264   ::webkit-scrollbar-thumb:horizontal {
265     background: #888888;
266   }
267
268   ::webkit-scrollbar-thumb:vertical {
269     background: #888888;
270   }
271
272   ::webkit-scrollbar-thumb:horizontal {
273     background: #888888;
274   }
275
276   ::webkit-scrollbar-thumb:vertical {
277     background: #888888;
278   }
279
280   ::webkit-scrollbar-thumb:horizontal {
281     background: #888888;
282   }
283
284   ::webkit-scrollbar-thumb:vertical {
285     background: #888888;
286   }
287
288   ::webkit-scrollbar-thumb:horizontal {
289     background: #888888;
290   }
291
292   ::webkit-scrollbar-thumb:vertical {
293     background: #888888;
294   }
295
296   ::webkit-scrollbar-thumb:horizontal {
297     background: #888888;
298   }
299
300   ::webkit-scrollbar-thumb:vertical {
301     background: #888888;
302   }
303
304   ::webkit-scrollbar-thumb:horizontal {
305     background: #888888;
306   }
307
308   ::webkit-scrollbar-thumb:vertical {
309     background: #888888;
310   }
311
312   ::webkit-scrollbar-thumb:horizontal {
313     background: #888888;
314   }
315
316   ::webkit-scrollbar-thumb:vertical {
317     background: #888888;
318   }
319
320   ::webkit-scrollbar-thumb:horizontal {
321     background: #888888;
322   }
323
324   ::webkit-scrollbar-thumb:vertical {
325     background: #888888;
326   }
327
328   ::webkit-scrollbar-thumb:horizontal {
329     background: #888888;
330   }
331
332   ::webkit-scrollbar-thumb:vertical {
333     background: #888888;
334   }
335
336   ::webkit-scrollbar-thumb:horizontal {
337     background: #888888;
338   }
339
340   ::webkit-scrollbar-thumb:vertical {
341     background: #888888;
342   }
343
344   ::webkit-scrollbar-thumb:horizontal {
345     background: #888888;
346   }
347
348   ::webkit-scrollbar-thumb:vertical {
349     background: #888888;
350   }
351
352   ::webkit-scrollbar-thumb:horizontal {
353     background: #888888;
354   }
355
356   ::webkit-scrollbar-thumb:vertical {
357     background: #888888;
358   }
359
360   ::webkit-scrollbar-thumb:horizontal {
361     background: #888888;
362   }
363
364   ::webkit-scrollbar-thumb:vertical {
365     background: #888888;
366   }
367
368   ::webkit-scrollbar-thumb:horizontal {
369     background: #888888;
370   }
371
372   ::webkit-scrollbar-thumb:vertical {
373     background: #888888;
374   }
375
376   ::webkit-scrollbar-thumb:horizontal {
377     background: #888888;
378   }
379
380   ::webkit-scrollbar-thumb:vertical {
381     background: #888888;
382   }
383
384   ::webkit-scrollbar-thumb:horizontal {
385     background: #888888;
386   }
387
388   ::webkit-scrollbar-thumb:vertical {
389     background: #888888;
390   }
391
392   ::webkit-scrollbar-thumb:horizontal {
393     background: #888888;
394   }
395
396   ::webkit-scrollbar-thumb:vertical {
397     background: #888888;
398   }
399
400   ::webkit-scrollbar-thumb:horizontal {
401     background: #888888;
402   }
403
404   ::webkit-scrollbar-thumb:vertical {
405     background: #888888;
406   }
407
408   ::webkit-scrollbar-thumb:horizontal {
409     background: #888888;
410   }
411
412   ::webkit-scrollbar-thumb:vertical {
413     background: #888888;
414   }
415
416   ::webkit-scrollbar-thumb:horizontal {
417     background: #888888;
418   }
419
420   ::webkit-scrollbar-thumb:vertical {
421     background: #888888;
422   }
423
424   ::webkit-scrollbar-thumb:horizontal {
425     background: #888888;
426   }
427
428   ::webkit-scrollbar-thumb:vertical {
429     background: #888888;
430   }
431
432   ::webkit-scrollbar-thumb:horizontal {
433     background: #888888;
434   }
435
436   ::webkit-scrollbar-thumb:vertical {
437     background: #888888;
438   }
439
440   ::webkit-scrollbar-thumb:horizontal {
441     background: #888888;
442   }
443
444   ::webkit-scrollbar-thumb:vertical {
445     background: #888888;
446   }
447
448   ::webkit-scrollbar-thumb:horizontal {
449     background: #888888;
450   }
451
452   ::webkit-scrollbar-thumb:vertical {
453     background: #888888;
454   }
455
456   ::webkit-scrollbar-thumb:horizontal {
457     background: #888888;

```

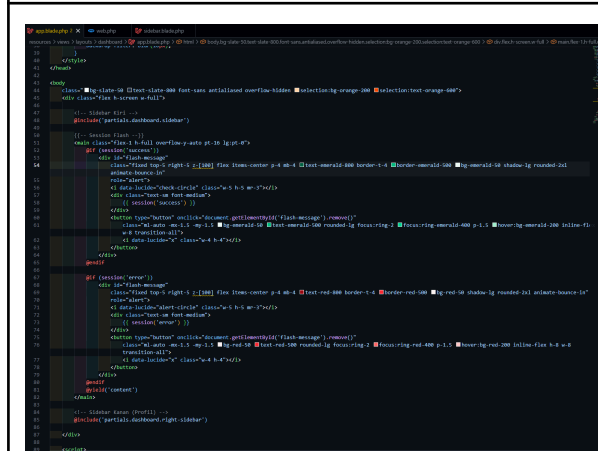
Pada langkah ini, saya menyusun kerangka dashboard yang berbeda dari halaman publik. Saya menyertakan sidebar kiri dan kanan menggunakan `@include`, serta area konten utama yang fleksibel melalui `@yield('content')`.

Ini adalah tampilan akhir dashboard manajemen ruang. Struktur tiga kolom (sidebar navigasi, konten utama, dan profil admin) berhasil dirender secara sempurna dengan tema warna yang konsisten.

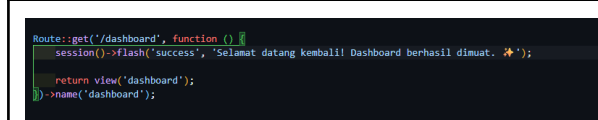
4. Tampilkan flash session message di layout: @if(session('success')) ... @endif



Halaman ini menampilkan contoh nyata flash message sukses yang muncul di pojok kanan atas setelah dashboard berhasil dimuat. Pesan melayang ini memberikan umpan balik instan kepada pengguna tanpa mengganggu konten utama.

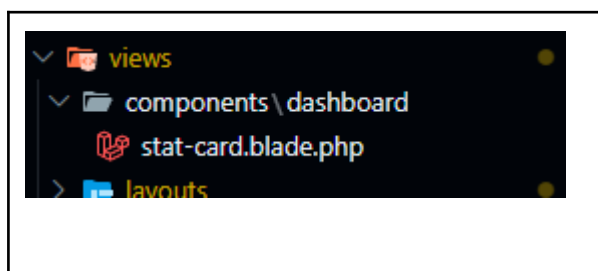


Pada langkahnya, saya menambahkan struktur kontrol @if(session('success')) di dalam file layout master. Kode ini berfungsi untuk mendeteksi adanya sesi "success" atau "error", lalu merender komponen peringatan secara dinamis dengan desain yang responsif.



Ini adalah route yang menunjukkan cara memicu pesan tersebut langsung melalui file route. Dengan menggunakan fungsi session()->flash(), sistem diperintahkan untuk mengirimkan pesan selamat datang satu kali (flash) tepat saat pengguna mengakses URL dashboard.

5. Buat komponen: resources/views/components/stat-card.blade.php (kartu statistik dengan props: judul, nilai, ikon, warna)



Saya membuat file stat-card.blade.php di dalam folder resources/views/components/dashboard agar komponen ini terorganisir khusus untuk kebutuhan antarmuka dashboard.

[illegible]

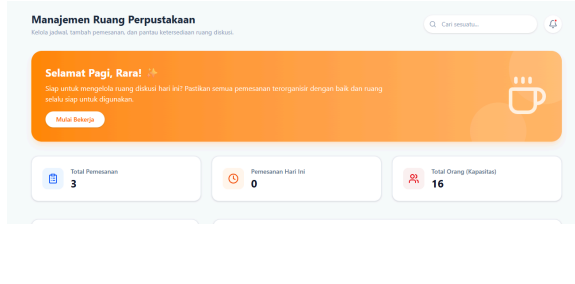
Di sini saya mendefinisikan `@props` untuk menerima data judul, nilai, ikon, dan warna. Saya juga menyertakan logika PHP sederhana untuk memetakan variasi warna (seperti biru, oranye, dan merah) agar tampilan ikon dan latar belakangnya serasi secara dinamis.

```

1 dashboard.blade.php | x
2 stat-card.blade.php
3 resources > views > dashboard.blade.php | x
4 man:hex-1h-full:overflow-y:auto;16:jspt-0" > div-p-6:md:slg:10:mar-w-7:dm:auto" > section.booking
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76

```

Gambar ini menunjukkan kemudahan penggunaan komponen dengan tag khusus `<x-dashboard.stat-card />`. Dengan cara ini, saya bisa memanggil kartu statistik berkali-kali hanya dengan mengganti nilai props-nya, sehingga kode di file dashboard utama menjadi jauh lebih bersih dan ringkas.

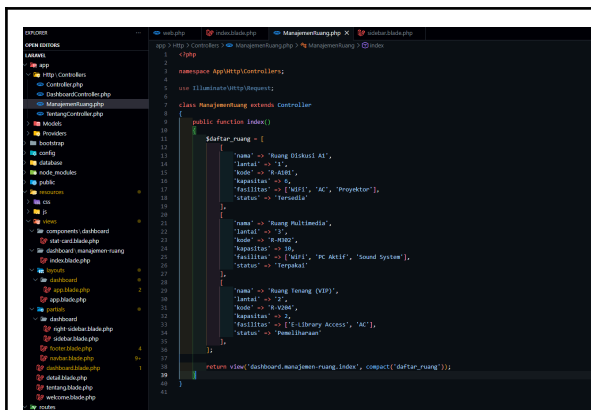


Ini adalah tampilan akhir di browser di mana tiga kartu statistik (Total Pemesanan, Hari Ini, dan Kapasitas) muncul berdampingan. Setiap kartu memiliki warna dan ikon yang berbeda sesuai dengan data yang dikirimkan.

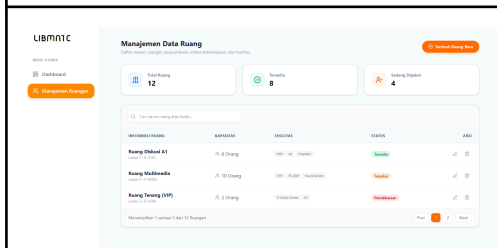
6. Gunakan @forelse di dashboard untuk loop array data dummy (jumlah mahasiswa, mata kuliah, dll)

[illegible]

Gambar ini menunjukkan penggunaan direktif `@forelse` di dalam tabel untuk melakukan perulangan pada array `$daftar_ruang`. Saya juga menyertakan blok `@empty` untuk menampilkan pesan khusus jika data ruangan belum tersedia, sehingga antarmuka tetap informatif.



Di sini saya mendefinisikan array data dummy \$daftar_ruang di dalam ManajemenRuangController. Setiap elemen array menyimpan informasi detail seperti nama ruang, kapasitas, fasilitas, dan status, yang kemudian dikirimkan ke view menggunakan fungsi compact().

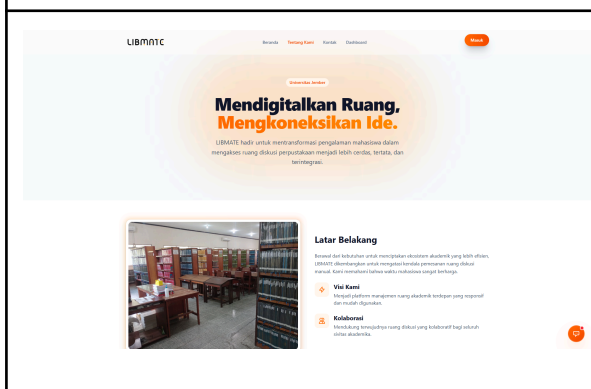


Hasil akhirnya adalah tabel manajemen ruang yang terisi secara otomatis berdasarkan data dari controller. Setiap baris menampilkan informasi secara terstruktur, termasuk penggunaan warna status yang berbeda dan badge fasilitas, memberikan tampilan profesional pada dashboard admin.

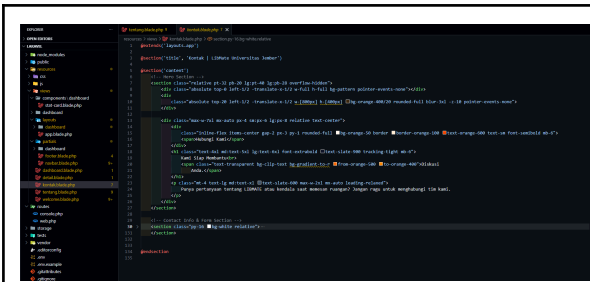
7. Buat halaman 'Tentang' dan 'Kontak' yang @extends layout yang sama



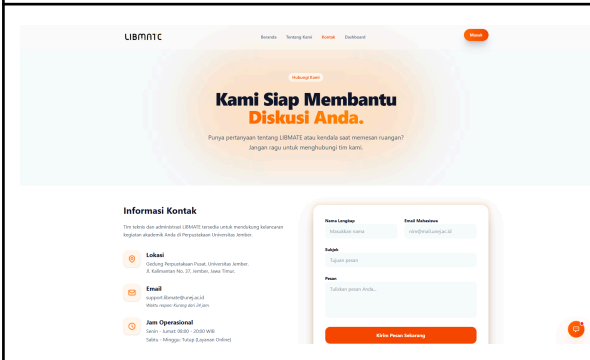
Gambar ini menunjukkan struktur kode tentang.blade.php yang menggunakan @extends('layouts.app'). Dengan mewarisi layout master yang sama, halaman ini dapat fokus sepenuhnya pada pengisian konten spesifik.



Ini adalah hasil render halaman "Tentang" yang menampilkan banner hero dan bagian latar belakang. Penggunaan layout master memastikan elemen global seperti navbar tetap konsisten, sementara konten utama menyajikan informasi mendalam mengenai filosofi layanan LibMate.

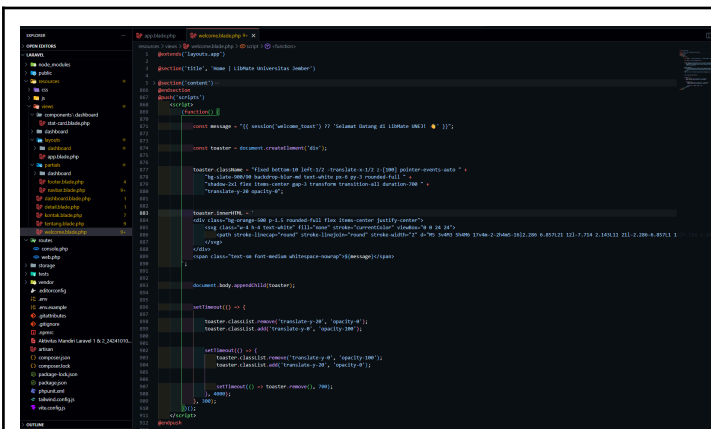


Serupa dengan halaman sebelumnya, kontak.blade.php juga menginduk pada layout master yang sama. Di sini saya menyusun formulir interaktif dan informasi detail hubungi kami menggunakan skema warna orange yang selaras, menjaga kesatuan desain di seluruh aplikasi.



Hasil akhir halaman "Kontak" menampilkan integrasi antara informasi lokasi, jam operasional, dan formulir pesan mahasiswa.

8. Gunakan @push('scripts') dan @stack('scripts') untuk menambahkan JavaScript spesifik per halaman



Langkah ini menunjukkan penggunaan direktif @push('scripts') di dalam file halaman tertentu. Di sini, saya menuliskan logika JavaScript murni untuk membuat elemen toaster secara dinamis, mengatur gaya visualnya, serta menentukan durasi animasi muncul dan menghilang otomatis.



Langkah kedua menampilkan hasil eksekusi script tersebut pada halaman utama. Ketika halaman berhasil dimuat, sebuah pesan sambutan melayang (toaster) muncul di bagian bawah layar tanpa memerlukan interaksi tombol.

AKTIVITAS MANDIRI 4 — Migration, Model & Seeder

□ AKTIVITAS MANDIRI 4 — Migration, Model & Seeder

1. Buat Model dan Migration sekaligus: `php artisan make:model Mahasiswa -m`
2. Edit file migration: tambahkan kolom nim, nama, email, jurusan (enum), ipk (decimal), semester, aktif (boolean), foto (nullable)
3. Jalankan migration: `php artisan migrate` — cek di phpMyAdmin bahwa tabel sudah terbuat
4. Edit Model Mahasiswa: isi \$fillable, tambahkan \$casts, dan buat local scope `scopeAktif()`
5. Buat seeder: `php artisan make:seeder MahasiswaSeeder` — isi dengan 5-10 data dummy
6. Daftarkan seeder di `DatabaseSeeder.php` dan jalankan: `php artisan db:seed`

7. Buka `php artisan tinker` dan coba query: `Mahasiswa::all()`, `Mahasiswa::where('jurusan','SI')->get()`, `Mahasiswa::avg('ipk')`
8. Coba buat relasi: buat tabel `mata_kuliahs` dan tabel pivot `krs`, lalu definisikan relasi `belongsToMany` di Model

□ *Tips: Di Tinker, bisa langsung uji query Eloquent tanpa membuat halaman. Sangat berguna untuk eksplorasi dan debugging.*

1. Buat Model dan Migration sekaligus: `php artisan make:model Ruangan -m`

Saya membuat untuk Ruangan

```
D:\PWEB-KULIAH\LARAVEL>php artisan make:model Ruangan -m
INFO Model [D:\PWEB-KULIAH\LARAVEL\app\Models\Ruangan.php] created successfully.
INFO Migration [D:\PWEB-KULIAH\LARAVEL\database\Migrations\2026_05_05_051502_create_ruangans_table.php] created successfully.
```

Langkah ini saya menjalankan perintah `php artisan make:model Ruangan -m`, sistem secara otomatis melakukan dua hal sekaligus:

1. Pembuatan Model: Menghasilkan file `Ruangan.php` di dalam folder `app/Models` yang berfungsi sebagai jembatan logika antara aplikasi dan tabel database.
2. Pembuatan Migration: Menghasilkan file cetak biru tabel `create_ruangans_table.php` di folder `database/migrations`.

2. Edit file migration: tambahkan kolom nim, nama, email, jurusan (enum), ipk (decimal), semester, aktif (boolean), foto (nullable)

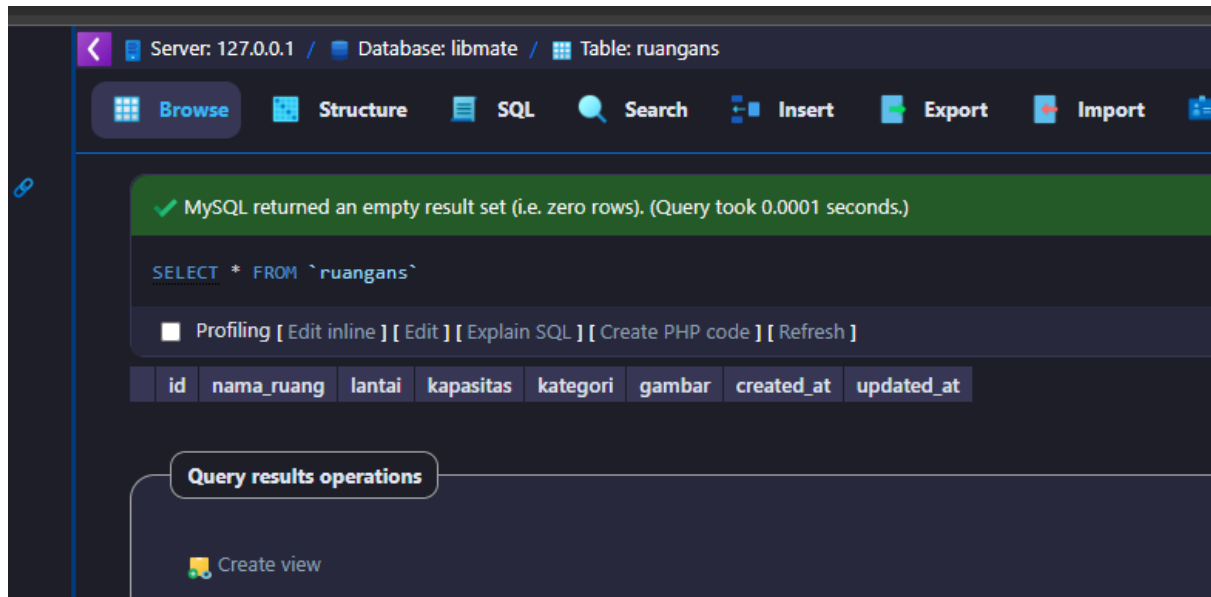
Setelah generate model dan migration, saya mengedit file create_ruangans_table.php untuk mendefinisikan atribut spesifik ruangan. Meskipun soal menggunakan contoh mahasiswa, saya menyesuaikan kolomnya agar relevan dengan proyek LibMate.

Detail Kolom yang Saya Tambahkan:

1. nama_ruang & kategori: Menggunakan tipe data string untuk identitas dan klasifikasi ruangan.
2. lantai & kapasitas: Didefinisikan sebagai string (yang nantinya di-cast ke integer) untuk menyimpan data numerik lokasi dan daya tampung.
3. gambar: Menggunakan tipe data string dengan tambahan properti nullable(), sesuai perintah soal agar kolom ini boleh dikosongkan jika foto belum tersedia.
4. timestamps(): Disertakan secara default untuk mencatat waktu pembuatan dan pembaruan data secara otomatis.

3. Jalankan migration: php artisan migrate — cek di phpMyAdmin bahwa tabel sudah terbuat

```
D:\PWEB-KULIAH\LARAVEL>php artisan migrate
INFO Running migrations.
2026_05_05_051502_create_ruangans_table ..... 118.75ms DONE
```

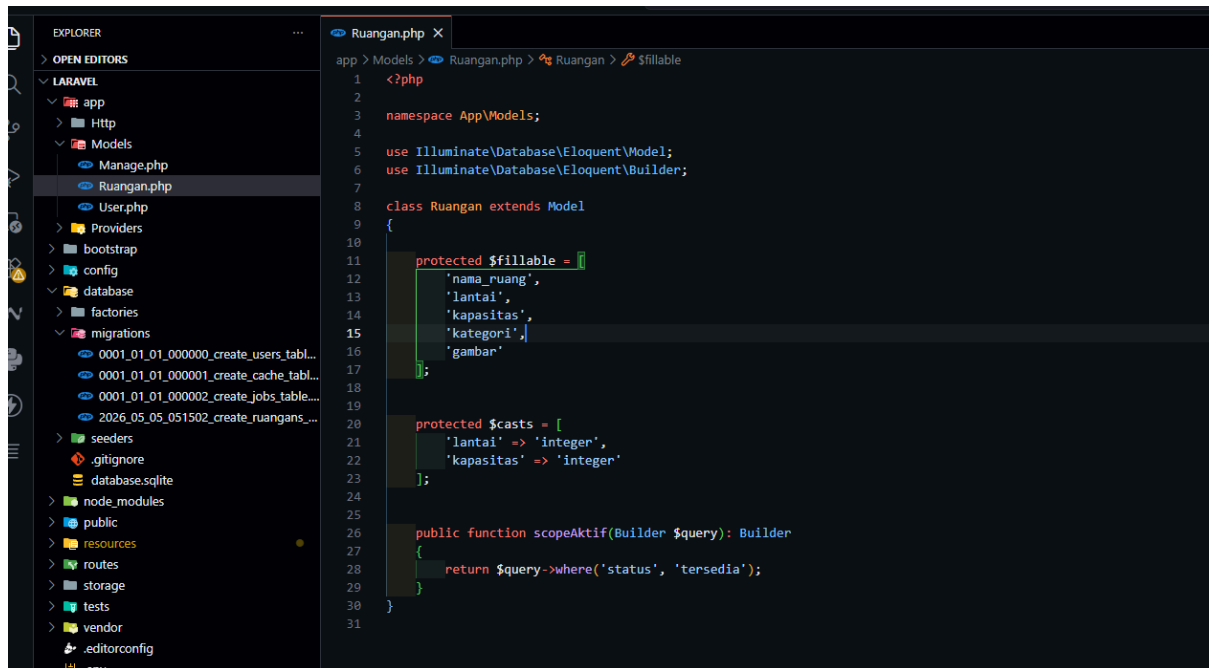


Setelah merancang struktur kolom, saya menjalankan perintah php artisan migrate di terminal untuk menerapkannya ke database.

1. Terminal: Status menunjukkan DONE untuk file create_ruangans_table, yang berarti tabel berhasil dibuat tanpa error.
2. phpMyAdmin: Saya melakukan verifikasi langsung pada database libmate. Seperti yang terlihat pada gambar, tabel ruangans sudah muncul dengan struktur kolom yang sesuai: id, nama_ruang, lantai, kapasitas, kategori, gambar, serta timestamps.

Saat ini tabel masih kosong (zero rows) dan siap untuk diisi dengan data dummy melalui proses seeding di langkah berikutnya.

4. Edit Model Mahasiswa: isi \$fillable, tambahkan \$casts, dan buat local scope scopeAktif()



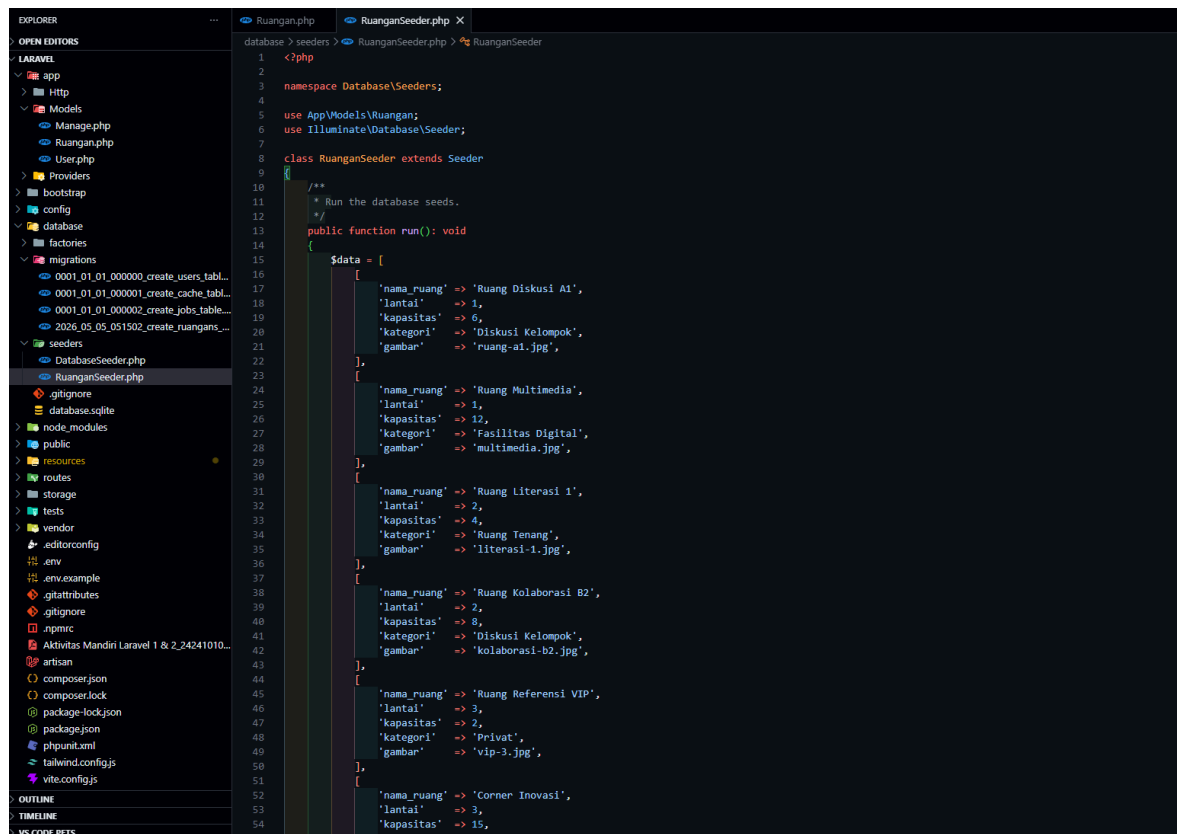
```
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6 use Illuminate\Database\Eloquent\Builder;
7
8 class Ruangan extends Model
9 {
10
11     protected $fillable = [
12         'nama_ruang',
13         'lantai',
14         'kapasitas',
15         'kategori',
16         'gambar'
17     ];
18
19     protected $casts = [
20         'lantai' => 'integer',
21         'kapasitas' => 'integer'
22     ];
23
24     public function scopeAktif(Builder $query): Builder
25     {
26         return $query->where('status', 'tersedia');
27     }
28 }
29
30
31
```

Pada tahap ini, saya melakukan optimasi pada file Ruangan.php sesuai instruksi, saya menambahkan tiga komponen utama:

1. \$fillable: Saya mendaftarkan kolom nama_ruang, lantai, kapasitas, kategori, dan gambar agar dapat diisi secara massal (mass assignment) dengan aman.
 2. \$casts: Saya menambahkan pengaturan casting untuk kolom lantai dan kapasitas agar secara otomatis dikonversi menjadi tipe data integer saat diakses di aplikasi.
 3. Local Scope scopeAktif(): Saya membuat scope kueri bernama aktif yang berfungsi untuk menyaring ruangan dengan status 'tersedia' secara instan.
5. Buat seeder: php artisan make:seeder MahasiswaSeeder — isi dengan 5-10 data dummy

```
D:\PWEB-KULIAH\LARAVEL>php artisan make:seeder RuanganSeeder
```

```
INFO Seeder [D:\PWEB-KULIAH\LARAVEL\database\seeders\RuanganSeeder.php] created successfully.
```



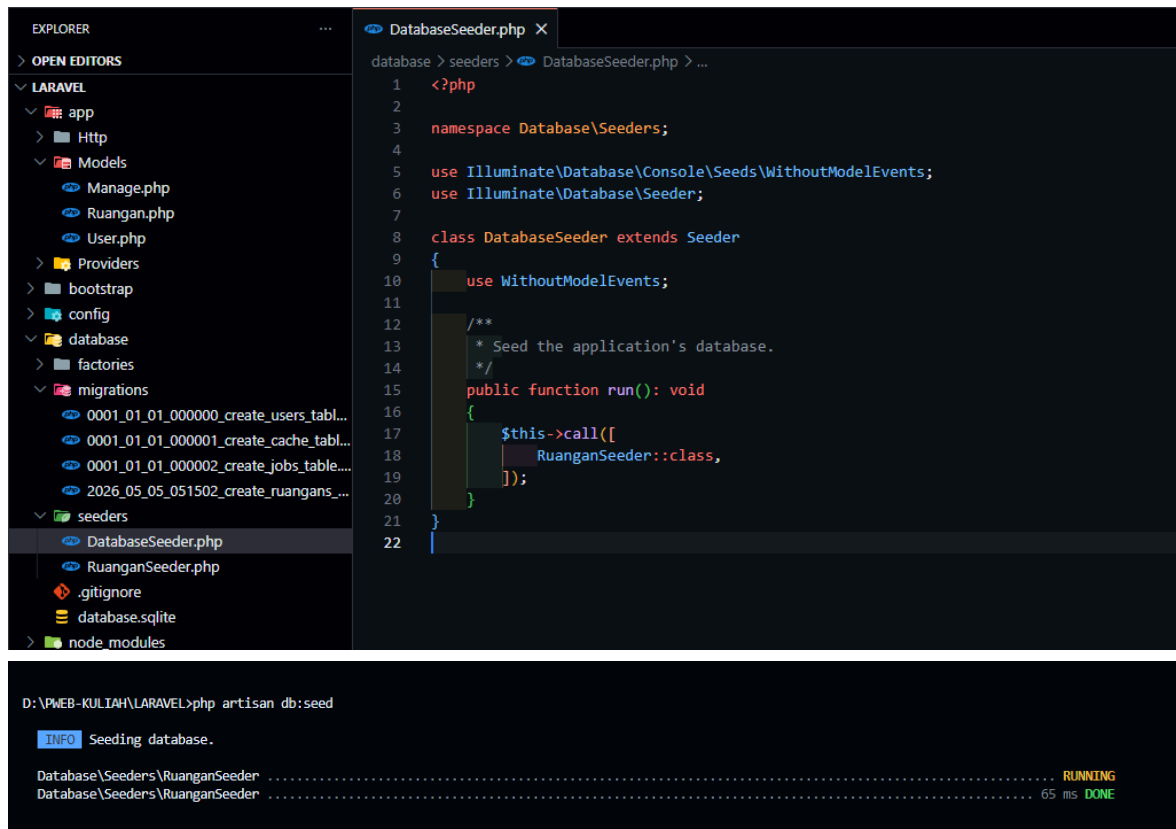
```
1 <?php
2
3 namespace Database\Seeders;
4
5 use App\Models\Ruang;
6 use Illuminate\Database\Seeder;
7
8 class RuanganSeeder extends Seeder
9 {
10     /**
11      * Run the database seeds.
12      */
13     public function run(): void
14     {
15         $data = [
16             [
17                 'nama_ruang' => 'Ruang Diskusi A1',
18                 'lantai' => 1,
19                 'kapasitas' => 6,
20                 'kategori' => 'Diskusi Kelompok',
21                 'gambar' => 'ruang-a1.jpg',
22             ],
23             [
24                 'nama_ruang' => 'Ruang Multimedia',
25                 'lantai' => 1,
26                 'kapasitas' => 12,
27                 'kategori' => 'Fasilitas Digital',
28                 'gambar' => 'multimedia.jpg',
29             ],
30             [
31                 'nama_ruang' => 'Ruang Literasi 1',
32                 'lantai' => 2,
33                 'kapasitas' => 4,
34                 'kategori' => 'Ruang Tenang',
35                 'gambar' => 'literasi-1.jpg',
36             ],
37             [
38                 'nama_ruang' => 'Ruang Kolaborasi B2',
39                 'lantai' => 2,
40                 'kapasitas' => 8,
41                 'kategori' => 'Diskusi Kelompok',
42                 'gambar' => 'kolaborasi-b2.jpg',
43             ],
44             [
45                 'nama_ruang' => 'Ruang Referensi VIP',
46                 'lantai' => 3,
47                 'kapasitas' => 2,
48                 'kategori' => 'Privat',
49                 'gambar' => 'vip-3.jpg',
50             ],
51             [
52                 'nama_ruang' => 'Corner Inovasi',
53                 'lantai' => 3,
54                 'kapasitas' => 15,
55                 'kategori' => 'Inovasi',
56             ]
57         ];
```

Langkah selanjutnya adalah mengisi tabel yang telah kita buat dengan data awal agar aplikasi bisa langsung diuji. Meskipun instruksi soal memberikan contoh MahasiswaSeeder, saya mengerjakannya untuk RuanganSeeder agar sesuai dengan kebutuhan proyek LibMate.

Tahapan yang saya lakukan:

1. Generate Seeder: Saya menjalankan perintah `php artisan make:seeder RuanganSeeder` untuk membuat file seeder baru di folder `database/seeds`.
2. Pengisian Data: Di dalam metode `run()`, saya menyusun array `$data` yang berisi 7 data dummy ruangan perpustakaan. Setiap data mencakup informasi lengkap seperti `nama_ruang`, `lantai`, `kapasitas`, `kategori`, dan nama file gambar.
3. Iterasi Data: Saya menggunakan perulangan `foreach` untuk memproses array tersebut dan memasukkannya ke database menggunakan perintah `Ruang::create($ruang)`.

6. Daftarkan seeder di DatabaseSeeder.php dan jalankan: php artisan db:seed



The image shows a code editor with the Explorer panel on the left and the DatabaseSeeder.php file open in the main editor. The Explorer panel shows the project structure, including the database directory and the seeders directory. The DatabaseSeeder.php file contains the following code:

```
1 <?php
2
3 namespace Database\Seeders;
4
5 use Illuminate\Database\Console\Seeds\WithoutModelEvents;
6 use Illuminate\Database\Seeder;
7
8 class DatabaseSeeder extends Seeder
9 {
10     use WithoutModelEvents;
11
12     /**
13      * Seed the application's database.
14      */
15     public function run(): void
16     {
17         $this->call([
18             RuanganSeeder::class,
19         ]);
20     }
21 }
22
```

The terminal window at the bottom shows the command `D:\PWEB-KULIAH\LARAVEL>php artisan db:seed` being executed. The output shows the status of the seeding process:

```
D:\PWEB-KULIAH\LARAVEL>php artisan db:seed
INFO Seeding database.
Database\Seeders\RuanganSeeder ..... RUNNING
Database\Seeders\RuanganSeeder ..... 65 ms DONE
```

Setelah menyusun data dummy di RuanganSeeder, langkah terakhir adalah mengintegrasikannya ke dalam sistem agar bisa dijalankan secara kolektif.

Langkah-langkah yang saya lakukan:

1. Registrasi Seeder: Saya membuka file DatabaseSeeder.php dan mendaftarkan RuanganSeeder::class di dalam metode run() menggunakan perintah `$this->call()`.
2. Eksekusi Perintah: Saya menjalankan perintah `php artisan db:seed` melalui terminal untuk memproses semua seeder yang telah didaftarkan.
3. Verifikasi Status: Terminal menunjukkan status **RUNNING** yang kemudian berubah menjadi **DONE** dalam waktu 65 ms, menandakan bahwa seluruh data ruangan berhasil dimasukkan ke dalam database tanpa kendala.

7. Buka php artisan tinker dan coba query: Mahasiswa::all(),
Mahasiswa::where('jurusan','SI')->get(), Mahasiswa::avg('ipk')

```
D:\PWEB-KULIAH\LARAVEL>php artisan tinker
Psy Shell v0.12.22 (PHP 8.4.13 - cli) by Justin Hileman
New PHP manual is available (latest: 3.0.5). Update with `doc --update-manual`
```

Mengaktifkan php artisan tinker

```
> App\Models\Ruangan::all();

= Illuminate\Database\Eloquent\Collection {#8092
  all: [
    App\Models\Ruangan {#8080
      id: 1,
      nama_ruang: "Ruang Tenang A",
      lantai: "1",
      kapasitas: "4",
      kategori: "Ruang Diskusi",
      gambar: "ruang1.jpeg",
      created_at: "2026-04-26 14:35:20",
      updated_at: "2026-04-26 14:35:20",
    },
    App\Models\Ruangan {#8079
      id: 2,
      nama_ruang: "Ruang Tenang B",
      lantai: "1",
      kapasitas: "6",
      kategori: "Ruang Diskusi",
      gambar: "ruang2.jpeg",
      created_at: "2026-04-26 14:35:20",
      updated_at: "2026-04-26 14:35:20",
    },
    App\Models\Ruangan {#8082
      id: 3,
      nama_ruang: "Ruang Meeting 1",
      lantai: "2",
      kapasitas: "10",
      kategori: "Ruang Meeting",
      gambar: "ruang3.jpg",
      created_at: "2026-04-26 14:35:20",
      updated_at: "2026-04-26 14:35:20",
    },
    App\Models\Ruangan {#8083
      id: 4,
      nama_ruang: "Ruang Meeting 2",
      lantai: "2",
      kapasitas: "12",
      kategori: "Ruang Meeting",
      gambar: "ruang5.jpg",
      created_at: "2026-04-26 14:35:20",
      updated_at: "2026-04-26 14:35:20",
    },
  ],
  App\Models\Ruangan {#8084
```

Saya menjalankan `Ruangan::all()` untuk menarik seluruh koleksi data dari tabel `ruangans`. Hasilnya menunjukkan semua data dummy yang sebelumnya dibuat lewat seeder berhasil dimuat dengan sempurna.

```
> App\Models\Ruangan::where('lantai', '1')->get();

= Illuminate\Database\Eloquent\Collection {#8115
  all: [
    App\Models\Ruangan {#8116
      id: 1,
      nama_ruang: "Ruang Tenang A",
      lantai: "1",
      kapasitas: "4",
      kategori: "Ruang Diskusi",
      gambar: "ruang1.jpeg",
      created_at: "2026-04-26 14:35:20",
      updated_at: "2026-04-26 14:35:20",
    },
    App\Models\Ruangan {#7354
      id: 2,
      nama_ruang: "Ruang Tenang B",
      lantai: "1",
      kapasitas: "6",
      kategori: "Ruang Diskusi",
      gambar: "ruang2.jpeg",
      created_at: "2026-04-26 14:35:20",
      updated_at: "2026-04-26 14:35:20",
    },
    App\Models\Ruangan {#8118
      id: 6,
      nama_ruang: "Ruang Diskusi A1",
      lantai: "1",
      kapasitas: "6",
      kategori: "Diskusi Kelompok",
      gambar: "ruang-a1.jpg",
      created_at: "2026-05-05 05:25:53",
      updated_at: "2026-05-05 05:25:53",
    },
    App\Models\Ruangan {#8119
      id: 7,
      nama_ruang: "Ruang Multimedia",
      lantai: "1",
      kapasitas: "12",
      kategori: "Fasilitas Digital",
      gambar: "multimedia.jpg",
      created_at: "2026-05-05 05:25:53",
      updated_at: "2026-05-05 05:25:53",
    },
  ],
}
```

Saya menggunakan `Ruangan::where('lantai', '1')->get()` untuk melakukan

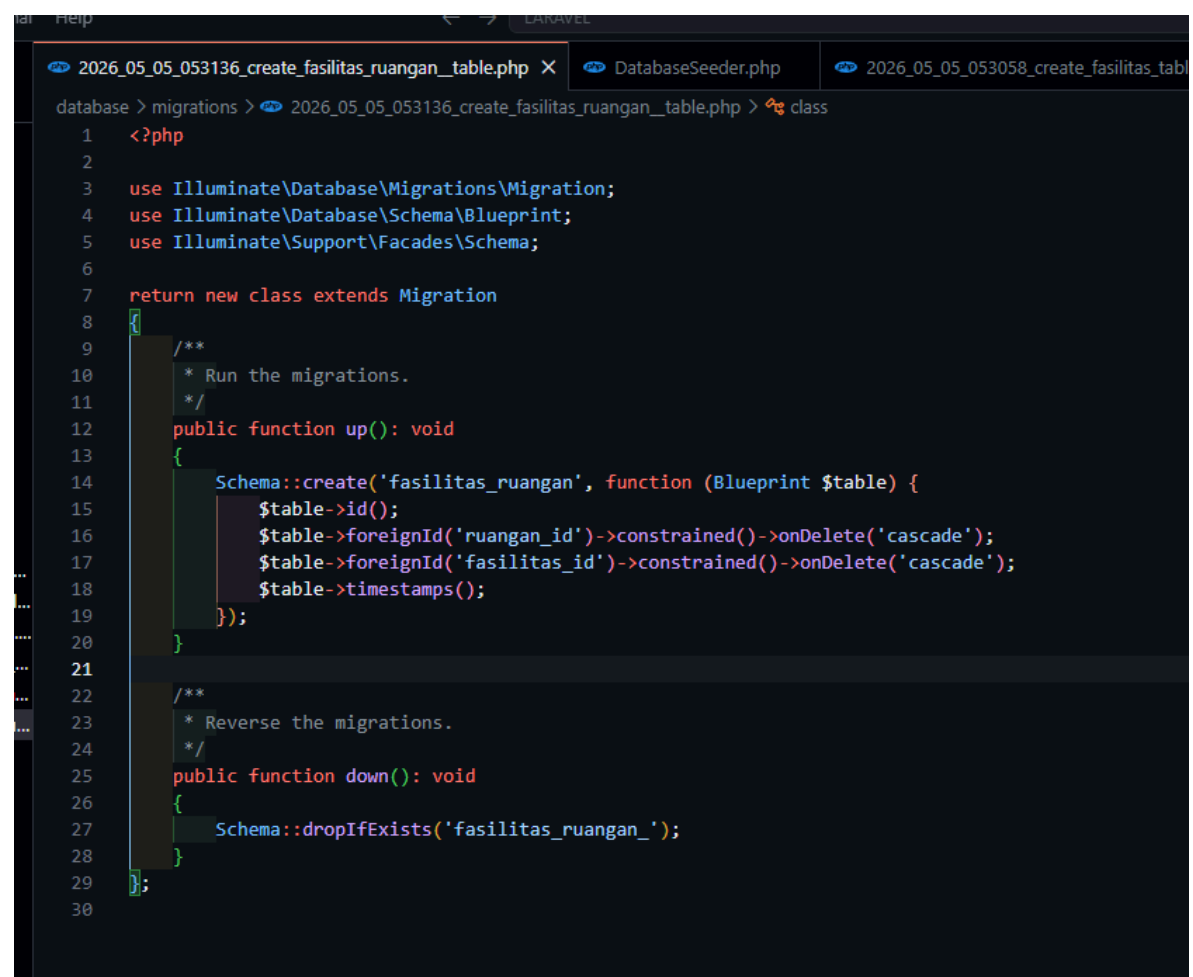
filter data. Tinker berhasil menampilkan hanya ruangan-ruangan yang berlokasi di lantai 1, seperti "Ruang Tenang A" dan "Ruang Tenang B".

```
> App\Models\Ruangan::avg('kapasitas');  
  
= 9.1666666666667
```

Saya menguji perhitungan rata-rata menggunakan `Ruangan::avg('kapasitas')`. Sistem secara otomatis menghitung seluruh nilai kapasitas dan menghasilkan angka rata-rata sebesar 9.16.

8. Coba buat relasi: buat tabel mata_kuliahs dan tabel pivot krs, lalu definisikan relasi belongsToMany di Model

Saya membuat untuk tabel fasilitas dan tabel pivot fasilitas ruangan



```
database > migrations > 2026_05_05_053136_create_fasilitas_ruangan_table.php > class  
1  <?php  
2  
3  use Illuminate\Database\Migrations\Migration;  
4  use Illuminate\Database\Schema\Blueprint;  
5  use Illuminate\Support\Facades\Schema;  
6  
7  return new class extends Migration  
8  {  
9      /**  
10       * Run the migrations.  
11       */  
12     public function up(): void  
13     {  
14         Schema::create('fasilitas_ruangan', function (Blueprint $table) {  
15             $table->id();  
16             $table->foreignId('ruangan_id')->constrained()->onDelete('cascade');  
17             $table->foreignId('fasilitas_id')->constrained()->onDelete('cascade');  
18             $table->timestamps();  
19         });  
20     }  
21  
22     /**  
23      * Reverse the migrations.  
24      */  
25     public function down(): void  
26     {  
27         Schema::dropIfExists('fasilitas_ruangan');  
28     }  
29 };  
30
```

2026_05_05_053136_create_fasilitas_ruangan_table.php

DatabaseSeeder.php

2026_05_05_053058_create_fasilitas_table.php X

database > migrations > 2026_05_05_053058_create_fasilitas_table.php > class > down

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12       public function up(): void
13       {
14           Schema::create('fasilitas', function (Blueprint $table) {
15               $table->id();
16               $table->string('nama_fasilitas'); // Contoh: WiFi, Proyektor, AC
17               $table->timestamps();
18           });
19       }
20
21       /**
22        * Reverse the migrations.
23        */
24       public function down(): void
25       {
26           Schema::dropIfExists('fasilitas');
27       }
28   };
29
```

```
help 2026_05_05_053136_create_fasilitas_ruangan_table.php DatabaseSeeder.php 2026_05
app > Models > Ruangan.php > Ruangan
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Builder;
7  use Illuminate\Database\Eloquent\Relations\BelongsTo;
8
9  class Ruangan extends Model
10 {
11
12     protected $fillable = [
13         'nama_ruang',
14         'lantai',
15         'kapasitas',
16         'kategori',
17         'gambar'
18     ];
19
20
21     protected $casts = [
22         'lantai' => 'integer',
23         'kapasitas' => 'integer'
24     ];
25
26
27     public function scopeAktif(Builder $query): Builder
28     {
29         return $query->where('status', 'tersedia');
30     }
31
32     public function fasilitas(): BelongsTo
33     {
34         return $this->belongsTo(Fasilitas::class, 'fasilitas_ruangan');
35     }
36 }
37
```



```
app > Models > Fasilitas.php > Fasilitas
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\BelongsToMany;
7
8  class Fasilitas extends Model
9  {
10     protected $fillable = ['nama_fasilitas'];
11
12
13     public function ruangs(): BelongsToMany
14     {
15         return $this->belongsToMany(Ruangan::class, 'fasilitas_ruangan');
16     }
17 }
18
```

Pada tahap ini, saya mengimplementasikan relasi di mana satu ruangan bisa memiliki banyak fasilitas, dan satu jenis fasilitas bisa tersedia di banyak ruangan. Meskipun instruksi soal mencontohkan mata kuliah, saya menyesuaikannya dengan kebutuhan fasilitas ruangan.

Langkah-langkah yang saya lakukan:

1. Pembuatan Tabel Master: Saya membuat migration `create_fasilitas_table` untuk menampung daftar fasilitas seperti WiFi, AC, dan Proyektor.
2. Pembuatan Tabel Pivot: Saya membuat tabel penghubung `fasilitas_ruangan`. Di dalam migration ini, saya mendefinisikan `foreignId` untuk `ruangan_id` dan `fasilitas_id` dengan tambahan `onDelete('cascade')` agar data penghubung otomatis terhapus jika data induk dihapus.
3. Definisi Relasi di Model: Saya menggunakan metode `belongsToMany()` pada Model `Ruangan` untuk menyambungkan kueri antar tabel secara otomatis.